Entrelíneas

Documentación del proyecto

Catálogo web, pedidos por WhatsApp, Mercado Pago, mostrador y caja

Proyecto de portfolio · Generado el 4 de octubre de 2026

Índice

1. Bitácora del proyecto
    - Estado actual
    - Entorno de desarrollo verificado
    - Sesiones
2. PRD — Requisitos del producto
    1. Resumen
    2. Problema
    3. Objetivos
    4. Usuarios
    5. Alcance del MVP
    6. Fuera del MVP (etapa 2 en adelante)
    7. Requisitos no funcionales
    8. Criterios de éxito de la demo
    9. Riesgos
3. Arquitectura
    1. Vista general
    2. Estructura del repositorio
    3. Modelo de datos
    4. Estados de una venta
    5. Flujos críticos
    6. Endpoints
    7. Autenticación
    8. Integraciones
    9. Variables de entorno
    10. Entornos y deploy
4. Arquitectura — Anexo: Prisma explicado
    1. Qué es Prisma y qué problema resuelve
    2. Las piezas y cómo se relacionan
    3. La conexión
    4. Escribir el esquema
    5. Generar el cliente
    6. Migraciones: cómo cambia la base sin perder datos
    7. Leer datos
    8. Escribir datos
    9. Dinero: el tipo Decimal
    10. Transacciones: todo o nada
    11. Errores de Prisma que vamos a manejar
    12. Dónde se usa Prisma dentro de la API

13. Tests contra la base de prueba
14. Datos de ejemplo (seed)
15. Prisma Studio
16. Resumen de comandos
17. Glosario

5. Diseño

1. Idea central
2. Color
3. Tipografía
4. Layouts
5. Componentes
6. Movimiento
7. Textos
8. Tokens para Tailwind

6. Reglas del proyecto

1. Generales
2. Idioma y nombres
3. Backend (Express)
4. Dinero, stock y caja (reglas críticas)
5. Mercado Pago
6. Frontend (Next.js)
7. Seguridad
8. Accesibilidad
9. Tests
10. Git

7. Tareas

Fase 0 — Preparación
Fase 1 — Base de datos y usuarios
Fase 2 — Libros y catálogo
Fase 3 — Carrito y pedidos
Fase 4 — Pagos
Fase 5 — Mostrador y caja
Fase 6 — Dashboard y facturas
Fase 7 — Calidad y publicación
Etapa 2 (después del MVP)

8. Memoria del proyecto

Contexto
Decisiones tomadas
Decisiones pendientes
Notas técnicas para recordar
Estado
Registro de sesiones

Bitácora del proyecto

Registro de avance, decisiones y observaciones. Se actualiza en cada sesión de trabajo y se regenera el PDF con `npm run docs:pdf` desde la raíz del repositorio.

Estado actual

Ítem	Valor
Fase	0 — Preparación (completa)
Rama de trabajo	`feat/fase-0-preparacion` → pull request a `main`
Repositorio	https://github.com/CyntiaCarrizo/Entrelineas
Tests	6 pasando (Vitest + Supertest)
Próximo paso	Fase 1: tablas con Prisma, seed, login y roles

Entorno de desarrollo verificado

Herramienta	Versión
Node.js	24.19.0 LTS (antes 20.19.5)
npm	11.17.0
MySQL Server	9.3.0 (local)
Git	2.46.2
GitHub CLI (`gh`)	No instalado (se usa `git` directamente)
Espacio libre en C:	~0,9 GB después de instalar (justo)
Sistema	Windows 11 Pro

Sesiones

2026-10-04 — Lectura de la documentación y armado del PDF

Qué se hizo

- Lectura completa de `PRD (1).md`, `arquitectura.md`, `rules.md`, `task.md` y `memory.md`.
- Se creó la carpeta `documentacion/` con un generador que une todos los documentos en un único PDF (`Entrelineas-Documentacion.pdf`).
- Se corrigió en `memory.md` una línea en blanco que partía la tabla de decisiones (las filas de Importación y Librería de Excel no se mostraban como tabla).

Observaciones sobre la documentación

1. **Falta `design.md`.** Lo citan `rules.md` (tokens de Tailwind), `task.md` (Fase 0) y `memory.md`. Hay que escribirlo antes de configurar Tailwind: paleta (papel, tinta, verde tela, ocre), tipografías (Literata + Public Sans) y el componente estante.
2. **Nombre del PRD.** El archivo se llama `PRD (1).md` pero los demás documentos lo citan como `PRD.md`. Conviene renombrarlo al copiarlo a `docs/`.
3. **Numeración en `arquitectura.md`.** En "Flujos críticos" la sección 5.5 (Importación) aparece antes que la 5.4 (Cierre de caja).
4. **Decisiones pendientes que bloquean:** acceso a datos (Prisma o `mysql2`), antes de la Fase 1; formato real del Excel de la librería, antes de la importación (Fase 2).

Pendiente

- Decidir Prisma o `mysql2`.
- Escribir `design.md`.
- Empezar la Fase 0.

2026-10-04 — Prisma, repositorio y estructura

Decisiones

- **Acceso a datos: Prisma ORM 7.10.** Se fija la 7.10 porque la etiqueta `latest` de npm apunta a Prisma 8 en versión candidata (rc). Se conecta a MySQL con `@prisma/adapter-mariadb`. La explicación completa está en el anexo "Prisma explicado", después del capítulo de Arquitectura.
- **Flujo de Git:**
  - `main` siempre estable y publicable.
  - Una rama por fase (`feat/fase-0-preparacion`, `feat/fase-1-base-de-datos`, …) que entra a `main` por *pull request*.
  - Cuando el catálogo esté estable (fin de la Fase 2) se conecta Vercel: publica `main` y genera una vista previa de cada rama.
  - La API y MySQL no pueden ir en Vercel: van en Railway o Render (`arquitectura.md`, sección 10).

Qué se hizo

- Documentos movidos a `docs/`. `PRD (1).md` pasó a llamarse `PRD.md`. Se sumó `design.md`, y el PDF usa ahora su paleta oficial.
- Repositorio Git iniciado en `main` con la documentación (primer commit). Se creó la rama `feat/fase-0-preparacion`.
- Raíz con *npm workspaces* (`apps/api`, `apps/web`, `docs/pdf`), `.gitignore` (excluye `.env`, claves y el cliente generado de Prisma) y `.gitattributes`.
- `apps/web` generado con `create-next-app` 16.3 (App Router, JavaScript, Tailwind v4).
- `apps/api` con `package.json`: Express 5, Zod 4, Prisma 7.10, Vitest y Supertest.
- `arquitectura.md`, `memory.md` y `task.md` actualizados con la decisión.

Problemas encontrados

1. **Sin espacio en disco.** El disco C: tiene 0,25 GB libres y `npm install` falla con `ENOSPC`. Next.js, Prisma y Vitest necesitan del orden de 1 GB entre dependencias y caché.
2. **Prisma 7 y JavaScript.** El generador nuevo (`prisma-client`) escribe el cliente en TypeScript. Node 20 no lo ejecuta directamente, y además dejó de tener soporte en abril de 2026. Propuesta: actualizar a Node 24 LTS, que ejecuta TypeScript simple de forma nativa. Se verifica al poder instalar.
3. **MySQL.** El usuario `root` pide contraseña. Hace falta para crear la base y el usuario `entrelineas` (guía de Prisma, sección 3.2).
4. **GitHub.** No está instalada la CLI `gh`. Para subir el repositorio hace falta crearlo en github.com (vacío, sin README) y pasar su URL.

Pendiente

- Liberar espacio en C: e instalar dependencias.
- Decidir la actualización a Node 24 LTS.
- Crear la base y el usuario de MySQL.
- Crear el repositorio en GitHub y hacer el primer *push*.
- Completar la Fase 0: estructura de Express, middleware de errores, Tailwind con los tokens, rewrite de `/api`, Vitest.

2026-10-04 — Fase 0 completa

Espacio en disco

- Se vació la papelera (19 elementos, 0,31 GB) y la caché de npm (0,9 GB). Pasamos de 0,25 GB a 1,36 GB libres.
- La caché de npm se vuelve a llenar en cada instalación: se limpió de nuevo después de instalar. Queda ~0,9 GB libre, que es poco. Convendría liberar más (`AppDataLocalTemp` ocupa 0,45 GB y `Descargas`, 0,47 GB).

Node.js 24 LTS

- Se instaló Node 24.19.0 con `winget`, reemplazando Node 20 en la misma carpeta. La raíz exige `"node": ">=24"`.
- Motivo: el cliente de Prisma 7 se genera en TypeScript (`client.ts`) y Node 24 lo ejecuta directamente (*type stripping*). Verificado: la API importa `client.ts` sin compilar.
- npm 11 bloquea los scripts de instalación. Se revisaron y aprobaron los de `prisma`, `@prisma/engines` (descarga el motor de migraciones) y `unrs-resolver` (binario de ESLint).

MySQL

- Se crearon tres bases: `entrelineas` (desarrollo), `entrelineas_shadow` (base sombra de Prisma) y `entrelineas_test` (tests).
- Usuario `entrelineas` con contraseña aleatoria guardada solo en `apps/api/.env`. Tiene permisos **solo sobre esas tres bases**. Primero se le había dado `CREATE, DROP` global para la base sombra,

pero se revocó: el servidor tiene bases de otros proyectos (`clinica_db`, `gym_db`, …) y ese permiso le habría permitido borrarlas.
- Problema: Prisma daba `pool timeout`. Probando el driver directamente apareció la causa: `ER_CANNOT_RETRIEVE_RSA_KEY`, por la autenticación de MySQL 9. Solución: `?allowPublicKeyRetrieval=true` en las URLs. Explicado en la guía de Prisma (sección 3.1).

API (`apps/api`)

- Express 5 con capas `routes → controllers → services → repositories`.
- `AppError` y middleware central de errores: respuestas `{ data }` o `{ error: { codigo, mensaje } }`, sin *stack traces*, con 404 y JSON mal formado.
- Middleware `validar(esquema)` con Zod.
- `src/db/prisma.js`: único cliente, con el adapter MariaDB. `prisma7.config.ts` con base sombra propia.
- Endpoints `GET /api/salud` y `GET /api/salud/db` (prueba la conexión). Script `npm run db:check`.
- Vitest + Supertest contra `entrelineas_test`: 6 tests pasando (salud, conexión a la base de prueba, 404, JSON inválido, validación).

Web (`apps/web`)

- Next.js 16.3 + Tailwind 4 con los tokens de `design.md` (`bg-papel`, `text-tinta`, `bg-tela`, `rounded-campo`, …).
- Literata y Public Sans con `next/font`. `lang="es-AR"`, foco visible y `prefers-reduced-motion`.
- Rewrite de `/api/*` hacia Express. Verificado de punta a punta: `localhost:3100/api/salud/db` → Express → MySQL.
- Página de inicio provisoria con el logo (línea ocre). `next build` y ESLint sin errores.

Incidente

- Al probar la API, el puerto 4000 ya estaba ocupado por la API de otro proyecto local (**bonhomia**). El comando que cerraba el servidor de prueba detuvo también ese proceso. Su `node --watch` lo reinició solo y se verificó que volvió a responder. Desde entonces, las pruebas usan los puertos 4100 y 3100, y se cierran solo esos.

GitHub

- Remoto: `https://github.com/CyntiaCarrizo/Entrelineas.git`. Se subieron `main` (documentación) y `feat/fase-0-preparacion`.

Pendiente

- Revisar y aprobar el *pull request* de `feat/fase-0-preparacion` a `main` en GitHub.
- Fase 1: modelos de Prisma para todas las tablas, primera migración, seed, login y roles.

PRD — Requisitos del producto

Documento de requisitos del producto. Define qué se construye, para quién y qué queda fuera del MVP.

1. Resumen

Entrelíneas es una librería ficticia de barrio que vende en su local y quiere vender online sin perder el trato personal con sus clientes. El sistema combina:

- Un **catálogo web público** donde cualquier persona arma un carrito sin crear cuenta.
- Un **checkout por WhatsApp**: el pedido se guarda en el sistema y se envía al WhatsApp de la librería, que lo confirma por chat.
- **Pagos** por Mercado Pago (Checkout Pro, confirmación automática) o transferencia (confirmación manual).
- Un **panel de administración** con pedidos, mostrador (punto de venta con lector de ISBN), caja diaria, carga de libros, usuarios y dashboard.

Es un proyecto de portfolio: tiene que funcionar de punta a punta en una demo pública, con datos de ejemplo y Mercado Pago en modo prueba.

2. Problema

Las librerías chicas venden por WhatsApp e Instagram de forma desordenada: el cliente pregunta si hay un libro, alguien revisa la estantería, se pasa el precio a mano, se cobra por transferencia y nadie sabe al final del día cuánto entró por cada medio. Las plataformas de e-commerce completas les resultan caras, impersonales y obligan al cliente a registrarse.

3. Objetivos

1. Que el cliente pueda ver el stock real y pedir en menos de un minuto, sin registrarse.
2. Que la librería centralice pedidos web y ventas del local en un solo sistema.
3. Que la caja del día muestre con exactitud cuánto entró por efectivo, transferencia y Mercado Pago, y por qué canal.
4. Que cargar un libro nuevo lleve segundos gracias al escáner y al autocompletado por ISBN.
5. (Portfolio) Demostrar integración real con Mercado Pago, manejo correcto de pagos (idempotencia, transacciones, anulaciones) y SEO con Next.js.

4. Usuarios

Usuario	Descripción	Acceso
Cliente	Persona que compra desde la web, generalmente desde el celular	Sin login
Vendedor	Atiende el mostrador, confirma pedidos, cobra, abre y cierra caja	Login, rol `vendedor`
Administrador	Dueño o encargado. Todo lo del vendedor más precios, usuarios, reportes y anulaciones	Login, rol `admin`

5. Alcance del MVP

5.1 Catálogo público

- Listado de libros con portada, título, autor, precio y disponibilidad.
- Búsqueda por título, autor e ISBN. Filtros por género y editorial. Paginación.
- Búsqueda y filtros reflejados en la URL (`/libros?genero=novela&q=borges`).
- Página de detalle por libro con URL amigable (`/libros/el-aleph-jorge-luis-borges`), sinopsis, datos y libros relacionados.
- Etiquetas Open Graph para que el link compartido por WhatsApp muestre portada, título y precio.
- Solo se muestran libros activos. Nunca se exponen datos internos.

5.2 Carrito y pedido por WhatsApp

- Carrito en el navegador. Datos del cliente: nombre y teléfono obligatorios, email opcional.
- Opción "Necesito factura" con CUIT/DNI, razón social y condición frente al IVA.
- Al confirmar: el pedido se guarda como `pendiente` con un código (`ENT-4821`) y un token público, y se abre WhatsApp con el mensaje armado.
- Protección antibots: Cloudflare Turnstile y límite de pedidos por IP.
- Los pedidos `pendiente` que nadie confirma vencen a las 2 horas (`expirada`).

5.3 Gestión de pedidos (admin)

- Lista con filtros por estado, búsqueda por código o teléfono.
- Confirmar pedido: descuenta stock y habilita el pago. Cancelar: devuelve el stock si ya se había descontado.
- Marcar como entregado o retirado.
- Copiar el link del pedido para enviarlo por WhatsApp.

5.4 Página pública del pedido

- URL única no adivinable (`/pedido/{token}`), sin login.
- Muestra estado, detalle y total.

- Si está confirmado: pagar con Mercado Pago, o ver los datos de transferencia (alias, CBU, monto exacto) y subir el comprobante.
- Si tiene factura cargada: botón "Descargar factura".

5.5 Pagos

- **Mercado Pago (Checkout Pro):** preferencia con `external_reference` = id de la venta; webhook que consulta el pago a la API y lo registra si está aprobado. Idempotente.
- **Transferencia:** número de operación obligatorio y único, comprobante opcional. Confirmación manual.
- **Efectivo, débito, crédito:** solo en el mostrador, registro manual.
- Un pago nunca se borra: se anula, registrando quién, cuándo y por qué.

5.6 Mostrador (punto de venta)

- Búsqueda por título o autor, o escaneo de ISBN (lector USB que funciona como teclado).
- Si el ISBN no existe, ofrecer darlo de alta.
- Cobro con uno o más métodos de pago. Stock descontado al registrar la venta.
- Requiere una caja abierta.

5.7 Caja diaria

- Apertura con efectivo inicial.
- Cada pago aprobado se asocia a la caja abierta en el momento del cobro, no a la fecha del pedido.
- Ingresos y egresos manuales (proveedor, insumos, retiro de efectivo).
- Cierre con arqueo: efectivo esperado contra contado, diferencia y observaciones.
- Resumen por método de pago y por canal. Historial de cajas cerradas.
- Los pagos de Mercado Pago que llegan sin caja abierta quedan "sin caja" y se asocian a la próxima caja que se abra.

5.8 Libros

- Alta escaneando el ISBN, con autocompletado desde Google Books y, si no hay resultado, Open Library.
- Portada copiada a Cloudinary (no se enlazan imágenes externas).
- Código interno para productos sin ISBN.
- Edición de precio, stock, género y estado (activo o inactivo).
- Cada cambio de stock queda registrado como movimiento.

5.9 Usuarios y roles

- Login con email y contraseña. Roles `admin` y `vendedor`.
- Cada venta, pago, movimiento y cierre de caja guarda el usuario que lo hizo.

5.10 Dashboard

- Ventas del día y del mes, por método de pago y por canal.
- Libros más vendidos. Pedidos pendientes de confirmar y de pago.

5.11 Facturas (carga manual)

- El admin sube el PDF de la factura emitida por la librería y lo asocia a la venta.
- La factura queda disponible en la página pública del pedido.
- La tabla de facturas ya contempla los campos de la futura integración con ARCA.

5.12 Importación desde Excel

- Pantalla del admin (solo rol `admin`) para subir el catálogo que la librería ya tiene en un archivo `.xlsx` o `.csv`.
- Plantilla descargable con las columnas recomendadas: ISBN, título, autor, editorial, género, precio, stock.
- Detección automática de columnas por el encabezado (por ejemplo "Título", "titulo" o "TITULO") y mapeo manual si no coincide.
- **Vista previa obligatoria** antes de guardar: cuántas filas se van a crear, cuántas actualizar, cuántas se omiten y cuáles tienen errores, con el motivo.
- Modo elegido explícitamente: solo crear libros nuevos, solo actualizar existentes, o ambos. Al actualizar, se elige qué campos se pisan (precio, stock, datos del libro).
- Los libros existentes se reconocen por ISBN o código interno.
- Cada cambio de stock por importación genera un movimiento de stock de tipo `importacion`.
- Informe final con descarga de las filas con error, para corregirlas y volver a importar.
- Opcional: completar datos faltantes y portadas desde Google Books / Open Library en segundo plano.
- Historial de importaciones: quién, cuándo, qué archivo y resultado.
- Script de línea de comandos con la misma lógica, para la carga inicial masiva o para cargar la demo.

6. Fuera del MVP (etapa 2 en adelante)

- Integración con ARCA en homologación (WSAA + WSFEv1, CAE, PDF con QR).
- QR de Mercado Pago integrado en el mostrador; Point integrado si hace falta.
- Encargos de libros sin stock.
- Historial de clientes por teléfono.
- Inventario avanzado: alertas de stock bajo, proveedores y compras.
- Promociones, descuentos y devoluciones.
- Exportación de reportes a Excel y PDF.
- Notificaciones por email.

7. Requisitos no funcionales

- **Mobile first** en el catálogo; el admin se usa principalmente en escritorio.
- **Rendimiento:** páginas del catálogo generadas en el servidor con revalidación; imágenes optimizadas por Cloudinary.
- **Seguridad:** contraseñas con bcrypt, sesión en cookie `httpOnly`, validación de entrada en todos los endpoints, firma del webhook validada, secretos solo en variables de entorno.
- **Integridad:** operaciones de dinero y stock dentro de transacciones; nada se borra en pagos, cajas ni ventas.
- **Accesibilidad:** contraste AA, foco visible, navegación por teclado (el mostrador debe poder usarse sin mouse).
- **SEO:** metadata por página, Open Graph, sitemap y URLs amigables.

8. Criterios de éxito de la demo

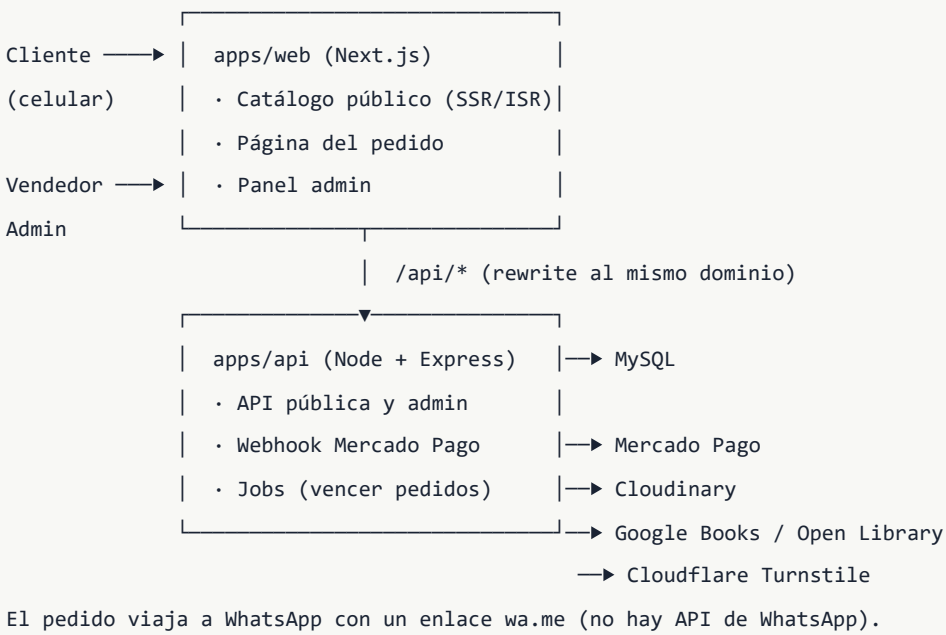
1. Un visitante pide un libro, paga con una tarjeta de prueba de Mercado Pago y el pedido aparece como pagado sin intervención manual.
2. Un vendedor abre caja, escanea un ISBN, cobra en efectivo y cierra caja con el arqueo correcto.
3. El link de un libro compartido por WhatsApp muestra la portada.
4. Los tests de caja, webhook y stock pasan.

9. Riesgos

Riesgo	Mitigación
Pedidos falsos o spam	Turnstile, límite por IP, confirmación manual por WhatsApp, vencimiento a 2 h
Webhook duplicado suma dos veces	Referencia única por pago y procesamiento idempotente
Venta simultánea del último ejemplar (web y mostrador)	Descuento de stock en transacción con verificación de cantidad
Cambios en APIs de Mercado Pago o de libros	Integraciones aisladas en servicios propios; revisar documentación en cada etapa
Excel del cliente desordenado (columnas distintas, ISBN convertidos a número, precios con formato)	Mapeo de columnas, normalización de datos, vista previa y descarga de errores
Alcance demasiado grande	MVP cerrado en este documento; todo lo demás va a etapa 2

Arquitectura

1. Vista general



El pedido viaja a WhatsApp con un enlace wa.me (no hay API de WhatsApp).

Decisiones principales

- **Next.js como frontend** (catálogo y admin) y **Express como API**. Express concentra toda la lógica de negocio, el acceso a datos y las integraciones. Next.js no accede a la base de datos.
- **Mismo dominio para web y API**: Next.js reescribe `/api/*` hacia Express (`rewrites` en `next.config.js`). Así la cookie de sesión es de primera parte y se evita configurar CORS con cookies entre dominios.
- **JavaScript** en ambos proyectos (coherente con el stack ya usado). Validación de datos con Zod.
- **Acceso a datos:** Prisma ORM 7 con el adapter `@prisma/adapter-mariadb` (compatible con MySQL). El esquema vive en `apps/api/prisma/schema.prisma` y las migraciones en `apps/api/prisma/migrations/`. Toda consulta vive en la capa `repositories/`. Explicación completa en el anexo `guia-prisma.md`.

2. Estructura del repositorio

```
entrelineas/
├── apps/
│   ├── web/                      # Next.js (App Router)
│   │   ├── app/
│   │   │   ├── layout.js
│   │   │   ├── page.js           # Inicio
│   │   │   ├── libros/
│   │   │   │   ├── page.js       # Listado con búsqueda y filtros
│   │   │   │   └── [slug]/page.js
│   │   │   ├── carrito/page.js
│   │   │   ├── pedido/[token]/page.js
│   │   │   └── admin/
│   │   │       ├── layout.js     # Verifica sesión, menú lateral
│   │   │       ├── page.js       # Dashboard
│   │   │       ├── pedidos/
│   │   │       ├── mostrador/
│   │   │       ├── caja/
│   │   │       ├── libros/
│   │   │       └── usuarios/
│   │   ├── components/
│   │   ├── context/              # Carrito (client)
│   │   └── lib/                  # fetch a la API, formato de precios
│   └── api/                      # Express
│       ├── prisma/               # schema.prisma y migraciones
│       └── src/
│           ├── routes/           # Definición de endpoints
│           ├── controllers/      # Leen request, llaman servicios, responden
│           ├── services/         # Lógica de negocio (caja, pagos, stock)
│           ├── repositories/     # Acceso a la base (único lugar que usa Prisma)
│           ├── integrations/     # mercadopago, cloudinary, libros, turnstile
│           ├── middlewares/      # auth, roles, rate limit, errores
│           ├── validators/       # Esquemas Zod
│           ├── jobs/             # Vencimiento de pedidos, enriquecimiento de importaciones
│           ├── importacion/      # Lectura de Excel/CSV, normalización, validación, ejecución
│           ├── scripts/          # importar-excel.js (línea de comandos)
│           └── app.js
├── docs/                         # PRD, arquitectura, rules, design, task, memory
└── README.md
```

3. Modelo de datos

Montos en `DECIMAL(12,2)`. Fechas en UTC. Ninguna tabla de dinero se borra físicamente.

usuarios

Campo	Tipo	Notas
id	INT PK	
nombre	VARCHAR	
email	VARCHAR UNIQUE	
password_hash	VARCHAR	bcrypt
rol	ENUM('admin','vendedor')	
activo	BOOLEAN	
creado_en	DATETIME	

generos

`id`, `nombre`, `slug` UNIQUE.

libros

Campo	Tipo	Notas
id	INT PK	
isbn	VARCHAR(13) UNIQUE NULL	Se guarda como texto
codigo_interno	VARCHAR UNIQUE NULL	Productos sin ISBN
slug	VARCHAR UNIQUE	`titulo-autor`, sin tildes
titulo, autor, editorial	VARCHAR	
genero_id	FK generos	
sinopsis	TEXT NULL	
paginas	INT NULL	
precio	DECIMAL(12,2)	
stock	INT	Nunca negativo
portada_public_id	VARCHAR NULL	Id de Cloudinary
activo	BOOLEAN	
creado_en, actualizado_en	DATETIME	

ventas

Representa tanto un pedido web como una venta del mostrador.

Campo	Tipo	Notas
id	INT PK	
codigo	VARCHAR UNIQUE	`ENT-4821`
token_publico	VARCHAR(32) UNIQUE NULL	Solo ventas web
canal	ENUM('web','local')	
estado	ENUM	Ver sección 4
cliente_nombre, cliente_telefono, cliente_email	VARCHAR NULL	
requiere_factura	BOOLEAN	
datos_fiscales	JSON NULL	cuit_dni, razon_social, condicion_iva
total	DECIMAL(12,2)	
usuario_id	FK usuarios NULL	Quién la registró o confirmó
stock_descontado	BOOLEAN	Para saber si cancelar devuelve stock
creado_en, confirmada_en, expira_en	DATETIME	

venta_items

`id`, `venta_id`, `libro_id`, `titulo` (copia), `precio_unitario` (copia), `cantidad`, `subtotal`. Los datos se copian para que un cambio de precio posterior no altere ventas pasadas.

cajas

Campo	Tipo	Notas
id	INT PK	
estado	ENUM('abierta','cerrada')	Una sola abierta a la vez
abierta_por, cerrada_por	FK usuarios	
abierta_en, cerrada_en	DATETIME	
efectivo_inicial	DECIMAL	
efectivo_esperado, efectivo_contado, diferencia	DECIMAL NULL	Se completan al cerrar
observaciones	TEXT NULL	

pagos

Campo	Tipo	Notas
id	INT PK	

Campo	Tipo	Notas
venta_id	FK ventas	
caja_id	FK cajas NULL	NULL = cobrado sin caja abierta
metodo	ENUM('efectivo','transferencia','mercadopago','debito','credito')	
monto	DECIMAL	
referencia	VARCHAR NULL	N.º de operación o id de pago de MP
comprobante_url	VARCHAR NULL	
estado	ENUM('pendiente','aprobado','rechazado','anulado')	
usuario_id	FK usuarios NULL	NULL si lo registró el webhook
anulado_por, anulado_en, motivo_anulacion		
creado_en	DATETIME	

Índice único: (`metodo`, `referencia`). Evita cargar dos veces la misma transferencia o el mismo pago de Mercado Pago.

movimientos_caja

`id`, `caja_id`, `tipo` ENUM('ingreso','egreso'), `concepto`, `monto`, `usuario_id`, `creado_en`, campos de anulación.

movimientos_stock

`id`, `libro_id`, `tipo` ENUM('alta','ingreso','venta','cancelacion','ajuste','importacion'), `cantidad` (con signo), `venta_id` NULL, `usuario_id` NULL, `motivo`, `creado_en`.

facturas

`id`, `venta_id`, `origen` ENUM('manual','arca'), `tipo` (A/B/C) NULL, `punto_venta` NULL, `numero` NULL, `cae` NULL, `cae_vencimiento` NULL, `pdf_url`, `usuario_id`, `creado_en`. Los campos de ARCA quedan vacíos en el MVP.

importaciones

`id`, `archivo_nombre`, `modo` ENUM('crear','actualizar','ambos'), `campos_actualizados` JSON, `estado` ENUM('vista_previa','procesando','completada','fallida'), `filas_total`, `creados`, `actualizados`, `omitidos`, `con_error`, `errores` JSON (fila, columna, motivo), `usuario_id`, `creado_en`, `finalizado_en`.

webhook_eventos

`id`, `proveedor`, `evento_id`, `payload` JSON, `procesado_en`, `error` NULL. Registro de auditoría de cada aviso recibido.

4. Estados de una venta

```
WEB:    pendiente ──▶ confirmada ──▶ comprobante_recibido ──▶ pagada ──▶ entregada
            │             │                                      ▲
            ▼             └──────────── (Mercado Pago) ──────────┘
         expirada         └──▶ cancelada (devuelve stock)

LOCAL:  pagada ──▶ entregada   (se crea ya pagada y entregada en el mostrador)
```

- `pendiente → expirada`: job cada 10 minutos para pedidos con `expira_en` vencido.
- `pendiente → confirmada`: acción del vendedor. Descuenta stock en transacción; si no alcanza, informa qué libros faltan.
- `confirmada → pagada`: webhook de Mercado Pago o confirmación manual de transferencia. Pasa a `pagada` cuando la suma de pagos aprobados cubre el total.

5. Flujos críticos

5.1 Pedido web

1. `POST /api/public/pedidos` con items, datos del cliente y token de Turnstile.
2. La API valida Turnstile, recalcula precios desde la base (nunca confía en precios del cliente), crea la venta `pendiente` con código, token y `expira_en`.
3. Devuelve el código y el texto del mensaje; el frontend abre `https://wa.me/{numero}?text=...`.

5.2 Pago con Mercado Pago

1. Desde `/pedido/{token}`, `POST /api/public/pedidos/{token}/mercadopago` crea la preferencia (`external_reference` = id de venta, `notification_url`, `back_urls`).
2. El cliente paga en Mercado Pago y vuelve a la página del pedido (solo informativo).
3. Mercado Pago llama a `POST /api/webhooks/mercadopago`. La API responde 200 de inmediato, valida la firma, guarda el evento y consulta el pago por id a la API de Mercado Pago.
4. Si está `approved`, en una transacción: verifica que no exista un pago con esa referencia, lo inserta en la caja abierta (o sin caja) y actualiza el estado de la venta.

5.3 Venta en el mostrador

1. El vendedor escanea o busca libros. `GET /api/admin/libros/isbn/{isbn}`.
2. `POST /api/admin/ventas` con items y pagos. En una transacción: verifica caja abierta y stock, crea venta `local`, items, pagos, movimientos de stock y descuenta stock.

5.5 Importación desde Excel

1. El admin sube el archivo: `POST /api/admin/importaciones/vista-previa` (máx. 5 MB, `.xlsx` o `.csv`, procesado en memoria, nunca se guarda en disco).
2. La API lee la primera hoja con ExcelJS, detecta columnas por encabezado y **normaliza** cada fila:
   - ISBN leído como texto; se quitan guiones y espacios; se corrige el formato científico (`9.78950E+12`) y se convierte ISBN-10 a ISBN-13; se valida el dígito verificador.
   - Precio: acepta `$15.000`, `15000`, `15.000,50`; se convierte a centavos.
   - Stock: entero mayor o igual a cero.
   - Texto: se recortan espacios y se unifica el género con los existentes.
3. Clasifica cada fila: `crear`, `actualizar`, `sin_cambios` o `error`, y detecta ISBN repetidos dentro del mismo archivo. Devuelve la vista previa y un id de importación en estado `vista_previa`.
4. El admin revisa, elige modo y campos a pisar y confirma: `POST /api/admin/importaciones/:id/confirmar`.
5. La API procesa en lotes de 200 filas, cada lote en una transacción: crea o actualiza libros, genera slugs y registra `movimientos_stock` de tipo `importacion`.
6. Guarda el resultado en `importaciones`. Si se pidió enriquecimiento, un job completa datos y portadas en segundo plano, respetando límites de las APIs externas.

5.4 Cierre de caja

`efectivo_esperado = efectivo_inicial + pagos en efectivo aprobados + ingresos manuales − egresos manuales`. Se guarda junto al contado y la diferencia. Una caja cerrada no admite cambios.

6. Endpoints

Públicos

Método	Ruta	Uso
GET	`/api/public/libros`	Listado con `q`, `genero`, `editorial`, `pagina`
GET	`/api/public/libros/:slug`	Detalle
GET	`/api/public/generos`	Filtros
POST	`/api/public/pedidos`	Crear pedido
GET	`/api/public/pedidos/:token`	Página del pedido
POST	`/api/public/pedidos/:token/mercadopago`	Crear preferencia
POST	`/api/public/pedidos/:token/comprobante`	Subir comprobante de transferencia
POST	`/api/webhooks/mercadopago`	Webhook

Admin (requieren sesión; marcados con ★ solo `admin`)

Método	Ruta	Uso
POST	`/api/auth/login`, `/api/auth/logout` · GET `/api/auth/me`	Sesión
GET, POST, PATCH	`/api/admin/libros`	ABM de libros (precio ★)
GET	`/api/admin/libros/isbn/:isbn`	Buscar en base o en APIs externas
GET	`/api/admin/pedidos`	Listado web
POST	`/api/admin/pedidos/:id/confirmar` · `/cancelar` · `/entregar`	Estados
POST	`/api/admin/ventas`	Venta de mostrador
POST	`/api/admin/pagos`	Registrar pago manual
POST	`/api/admin/pagos/:id/anular` ★	Anular pago
POST	`/api/admin/cajas/abrir` · GET `/api/admin/cajas/actual` · POST `/api/admin/cajas/cerrar`	Caja
GET	`/api/admin/cajas`	Historial
POST	`/api/admin/cajas/movimientos`	Ingreso o egreso manual
POST	`/api/admin/ventas/:id/factura`	Subir PDF
GET	`/api/admin/dashboard`	Métricas
GET	`/api/admin/importaciones/plantilla` ★	Descargar plantilla `.xlsx`
POST	`/api/admin/importaciones/vista-previa` ★	Subir archivo y obtener vista previa
POST	`/api/admin/importaciones/:id/confirmar` ★	Ejecutar la importación
GET	`/api/admin/importaciones` ★ · `/:id` · `/:id/errores`	Historial, detalle y descarga de errores
GET, POST, PATCH	`/api/admin/usuarios` ★	Usuarios

7. Autenticación

- Login devuelve una cookie `httpOnly`, `Secure`, `SameSite=Lax` con un JWT firmado (expira en 12 h).
- Middleware `requireAuth` y `requireRole('admin')` en Express.
- `app/admin/layout.js` llama a `/api/auth/me` y redirige a `/admin/login` si no hay sesión.

8. Integraciones

Servicio	Uso	Ubicación
Mercado Pago (SDK `mercadopago`)	Checkout Pro y consulta de pagos	`integrations/mercadopago.js`
Cloudinary	Portadas, comprobantes y facturas	`integrations/cloudinary.js`
Google Books / Open Library	Datos por ISBN	`integrations/libros.js`
Cloudflare Turnstile	Captcha del pedido	`integrations/turnstile.js`
ExcelJS	Lectura de `.xlsx` / `.csv` y generación de plantilla e informe de errores	`importacion/lector.js`
WhatsApp	Enlace `wa.me` armado en el frontend	`apps/web/lib/whatsapp.js`

9. Variables de entorno

apps/api: `DATABASE_URL`, `JWT_SECRET`, `MP_ACCESS_TOKEN`, `MP_WEBHOOK_SECRET`, `PUBLIC_URL`, `CLOUDINARY_URL`, `GOOGLE_BOOKS_KEY`, `TURNSTILE_SECRET`, `WHATSAPP_NUMERO`, `TRANSFERENCIA_ALIAS`, `TRANSFERENCIA_CBU`.

apps/web: `API_URL`, `NEXT_PUBLIC_SITE_URL`, `NEXT_PUBLIC_TURNSTILE_SITE_KEY`, `NEXT_PUBLIC_WHATSAPP_NUMERO`, `NEXT_PUBLIC_CLOUDINARY_CLOUD`.

10. Entornos y deploy

- **Desarrollo:** MySQL local, API en `localhost:4000`, web en `localhost:3000`, túnel (ngrok o Cloudflare Tunnel) para el webhook.
- **Demo:** web en Vercel; API y MySQL en Railway o Render. Mercado Pago con credenciales de prueba.
- **Etapa 2 (ARCA):** módulo `integrations/arca/` con WSAA (token cacheado) y WSFEv1, contra homologación.

Arquitectura — Anexo: Prisma explicado

Esta guía explica Prisma desde cero y con ejemplos de este proyecto: cómo se conecta a MySQL, cómo se escribe el esquema, cómo se consultan y modifican datos, y los conceptos que hay que tener claros para no romper el dinero ni el stock. Está pensada para leerse de corrido la primera vez y después usarse como consulta.

Versión usada: **Prisma ORM 7.10** (la última estable al empezar el proyecto) con MySQL 9.3 y Node.js 24.

1. Qué es Prisma y qué problema resuelve

Una aplicación en Node.js no "habla" MySQL directamente: necesita armar consultas SQL como texto, mandarlas a la base y convertir las filas que vuelven en objetos de JavaScript. Hacerlo a mano funciona, pero es fácil equivocarse en el nombre de una columna, olvidarse de un campo o escribir una consulta vulnerable.

Prisma es un ORM (*Object-Relational Mapper*, mapeador objeto-relacional): una herramienta que hace de traductor entre las tablas de la base y los objetos de JavaScript. En vez de escribir:

```
SELECT id, titulo, precio FROM libros WHERE activo = 1 ORDER BY titulo LIMIT 20;
```

se escribe:

```
const libros = await prisma.libro.findMany({
  where: { activo: true },
  select: { id: true, titulo: true, precio: true },
  orderBy: { titulo: 'asc' },
  take: 20,
});
```

y Prisma genera el SQL, lo ejecuta y devuelve un arreglo de objetos.

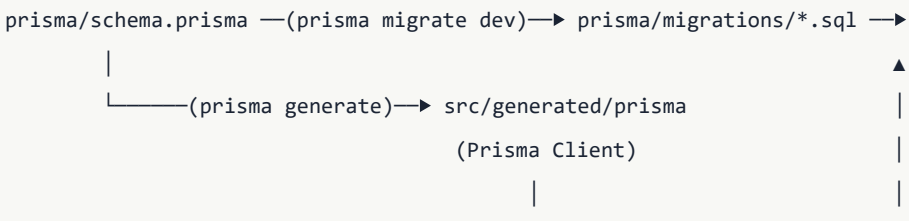
Por qué lo elegimos para Entrelíneas

Ventaja	Qué significa en el proyecto
Un solo archivo describe toda la base	`schema.prisma` es la fuente de verdad del modelo de datos de `arquitectura.md`
Migraciones incluidas	Cada cambio en la base queda versionado en Git como un archivo SQL
Consultas parametrizadas siempre	Protección contra inyección SQL sin pensarlo
Transacciones simples	Clave para la caja, los pagos y el stock

Ventaja	Qué significa en el proyecto
Prisma Studio	Una interfaz web para ver y editar datos durante el desarrollo

Qué no hace Prisma por nosotros: no decide las reglas del negocio. Que el stock nunca quede negativo o que un pago no se registre dos veces sigue siendo responsabilidad nuestra (ver secciones 9 y 10).

2. Las piezas y cómo se relacionan



Pieza	Qué es	Dónde vive
Esquema	Archivo que describe tablas, columnas y relaciones con el lenguaje de Prisma	`apps/api/prisma/schema.prisma`
Configuración	Le dice a la CLI de Prisma dónde está el esquema, dónde guardar las migraciones y a qué base conectarse	`apps/api/prisma7.config.ts`
CLI (`prisma`)	Programa de línea de comandos: crea migraciones, genera el cliente, abre Studio. Solo se usa en desarrollo y en el deploy	Dependencia de desarrollo
Migraciones	Archivos `.sql` que Prisma escribe a partir de los cambios del esquema	`apps/api/prisma/migrations/`
Prisma Client	Código generado a medida del esquema: `prisma.libro` , `prisma.venta` , etc.	`apps/api/src/generated/prisma/` (no se sube a Git; se regenera)
Driver adapter	El conector real con MySQL. Prisma 7 lo exige: el cliente ya no trae su propio motor, usa un driver de JavaScript	Paquete `@prisma/adapter-mariadb`

El adapter se llama "MariaDB" porque usa el driver `mariadb` de npm, que habla el mismo protocolo que MySQL. Es el adapter oficial de Prisma para ambas bases.

3. La conexión

3.1 La cadena de conexión

MySQL necesita cinco datos para aceptar una conexión: servidor, puerto, usuario, contraseña y base. Se escriben juntos en una URL:

```
mysql://USUARIO:CONTRASEÑA@SERVIDOR:PUERTO/BASE
mysql://entrelineas:clave-segura@localhost:3306/entrelineas?allowPublicKeyRetrieval=true
```

Lo que va después del `?` son **opciones** de la conexión. `allowPublicKeyRetrieval=true` hace falta con MySQL 8 y 9: el método de autenticación por defecto (`caching_sha2_password`) necesita la clave pública del servidor, y sin esta opción el driver no la pide y la conexión falla (en Prisma aparece como `pool timeout`). En local es seguro, porque la conexión no sale de la máquina. En la demo se va a usar una conexión con SSL.

Si la contraseña tiene caracteres especiales (`@`, `:`, `/`, `#`), hay que codificarlos (`@` se escribe `%40`), porque si no rompen la URL.

Esa URL va en el archivo `.env` de `apps/api`, que **nunca se sube a Git** (está en `.gitignore`). En el repositorio solo va `.env.example`, con la misma variable y un valor de ejemplo:

```
# apps/api/.env.example
DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas?allowPublicKeyRetrieval=true"
SHADOW_DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas_shadow?allowPublicKeyRetrie
TEST_DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas_test?allowPublicKeyRetrieval=
```

Son tres bases: la de desarrollo, la **sombra**, que Prisma usa para las migraciones (sección 6.3), y la de **tests**, que se puede vaciar sin miedo.

En el deploy (Railway o Render), la misma variable se carga en el panel del proveedor.

3.2 Usuario propio para la aplicación

No conviene que la aplicación use `root`. Se crea un usuario solo para el proyecto, con permisos **únicamente sobre sus tres bases**:

```
CREATE DATABASE entrelineas        CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE entrelineas_shadow CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE entrelineas_test   CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'entrelineas'@'localhost' IDENTIFIED BY 'clave-segura';
GRANT ALL PRIVILEGES ON entrelineas.*        TO 'entrelineas'@'localhost';
GRANT ALL PRIVILEGES ON entrelineas_shadow.* TO 'entrelineas'@'localhost';
GRANT ALL PRIVILEGES ON entrelineas_test.*   TO 'entrelineas'@'localhost';
```

Esto sigue el **principio de mínimo privilegio**: si la aplicación tuviera un error o una vulnerabilidad, no podría tocar otras bases del mismo servidor. Por eso no se le da el permiso global `CREATE, DROP ON *.*`, que Prisma necesitaría para crear la base sombra por su cuenta. En cambio, la creamos nosotros y se la indicamos en la configuración.

`utf8mb4` permite guardar cualquier carácter (tildes, eñes, emojis en un nombre de cliente). Es importante para una librería en español.

3.3 La configuración de la CLI

```
// apps/api/prisma7.config.ts
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: { path: 'prisma/migrations' },
  datasource: {
    url: process.env.DATABASE_URL,
    shadowDatabaseUrl: process.env.SHADOW_DATABASE_URL,
  },
});
```

- `import 'dotenv/config'` lee el archivo `.env` y carga sus valores en `process.env`.
- Este archivo lo usa **solo la CLI** (para migrar, generar, abrir Studio). La aplicación en ejecución se conecta por su cuenta (siguiente punto).
- A partir de Prisma 7.10 el archivo se llama `prisma7.config.ts` para no confundirse con el formato de Prisma 8.
- Es el único archivo TypeScript que escribimos en el proyecto. Prisma lo lee por su cuenta, así que no hace falta saber TypeScript: es solo un objeto de configuración.

3.4 Un solo cliente para toda la API

Crear un `PrismaClient` abre un **pool de conexiones**: un grupo de conexiones a MySQL que se reutilizan entre pedidos HTTP, porque abrir una conexión nueva cada vez es lento. Por eso se crea **una sola vez** y se importa desde donde haga falta:

```
// apps/api/src/db/prisma.js
import { PrismaMariaDb } from '@prisma/adapter-mariadb';
import { PrismaClient } from '../generated/prisma/client.ts';

const adapter = new PrismaMariaDb(process.env.DATABASE_URL);

export const prisma = new PrismaClient({ adapter });
```

Dos detalles de este archivo:

- El cliente generado está en TypeScript (`client.ts`). Node.js 24 lo ejecuta directamente: le quita los tipos al cargarlo (*type stripping*), sin compilar nada. Por eso el proyecto exige Node 24; con

Node 20 este import falla.

- `PrismaMariaDb` recibe la misma `DATABASE_URL` que usa la CLI, así hay una sola fuente de verdad para la conexión.

Al apagar el servidor se cierra el pool con `await prisma.$disconnect()`, para no dejar conexiones colgadas. En el proyecto lo hace `src/server.js` cuando recibe `Ctrl+C`.

3.5 Probar que conecta

```
await prisma.$queryRaw`SELECT 1`;
```

Si la URL, el usuario o la contraseña están mal, esta línea falla con un error que lo dice. En el proyecto lo hace el script `npm run db:check`.

4. Escribir el esquema

El esquema usa un lenguaje propio de Prisma, simple y declarativo. Tiene tres tipos de bloques: `generator` (qué cliente generar), `datasource` (qué base) y `model` (cada tabla).

```
// apps/api/prisma/schema.prisma
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "mysql"
}
```

4.1 Un modelo = una tabla

Ejemplo real del proyecto, la tabla `generos`:

```
model Genero {
  id     Int     @id @default(autoincrement())
  nombre String  @db.VarChar(80)
  slug   String  @unique @db.VarChar(80)
  libros Libro[]

  @@map("generos")
}
```

Cómo se lee cada línea: **nombre del campo**, **tipo** y **atributos** (lo que empieza con `@`).

Escritura	Significado
`Int`, `String`, `Boolean`, `DateTime`, `Decimal`, `Json`	Tipos de Prisma. Cada uno se traduce a un tipo de MySQL
`String?`	El `?` significa que acepta `NULL` (opcional)
`@id`	Clave primaria
`@default(autoincrement())`	MySQL asigna el número solo (`AUTO_INCREMENT`)
`@default(now())`	Toma la fecha y hora del momento de crear la fila
`@updatedAt`	Prisma la actualiza en cada modificación
`@unique`	No puede haber dos filas con el mismo valor
`@db.VarChar(80)`, `@db.Decimal(12, 2)`, `@db.Text`	Tipo exacto de MySQL, cuando el de Prisma no alcanza
`@map("columna")` / `@@map("tabla")`	Nombre real en la base (ver 4.2)
`@@unique([a, b])`	Unicidad combinada de dos columnas
`@@index([a])`	Índice para acelerar búsquedas

Los atributos con una `@` se aplican a un campo; los de dos `@@`, a todo el modelo.

4.2 Nombres: snake_case en la base, camelCase en JavaScript

`rules.md` pide tablas y columnas en `snake_case` (`creado_en`), y en JavaScript se usa `camelCase` (`creadoEn`). Prisma resuelve las dos cosas con `@map` y `@@map`: el modelo se llama `Libro` y la tabla `libros`; el campo es `creadoEn` y la columna `creado_en`.

```
model Libro {
  id               Int      @id @default(autoincrement())
  isbn             String?  @unique @db.VarChar(13)
  codigoInterno    String?  @unique @map("codigo_interno") @db.VarChar(40)
  slug             String   @unique @db.VarChar(200)
  titulo           String   @db.VarChar(255)
  autor            String   @db.VarChar(255)
  editorial        String   @db.VarChar(150)
  generoId         Int      @map("genero_id")
  genero           Genero   @relation(fields: [generoId], references: [id])
  sinopsis         String?  @db.Text
  paginas          Int?
  precio           Decimal  @db.Decimal(12, 2)
  stock            Int      @default(0)
  portadaPublicId  String?  @map("portada_public_id") @db.VarChar(255)
  activo           Boolean  @default(true)
  creadoEn         DateTime @default(now()) @map("creado_en")
  actualizadoEn    DateTime @updatedAt @map("actualizado_en")

  @@map("libros")
}
```

Notar que `isbn` es `String` y no un número: así lo pide `arquitectura.md`. Un ISBN puede empezar con cero y no se hacen cuentas con él.

4.3 Relaciones

Una relación conecta dos modelos. En Entrelíneas la más importante es **una venta tiene muchos ítems**:

```
model Venta {
  id     Int         @id @default(autoincrement())
  codigo String      @unique @db.VarChar(12)
  total  Decimal     @db.Decimal(12, 2)
  items  VentaItem[]              // lado "muchos": no crea columna
  pagos  Pago[]
  // …resto de los campos

  @@map("ventas")
}

model VentaItem {
  id             Int     @id @default(autoincrement())
  ventaId        Int     @map("venta_id")
  venta          Venta   @relation(fields: [ventaId], references: [id])
  libroId        Int     @map("libro_id")
  libro          Libro   @relation(fields: [libroId], references: [id])
  titulo         String  @db.VarChar(255)       // copia, ver arquitectura.md
  precioUnitario Decimal @map("precio_unitario") @db.Decimal(12, 2)
  cantidad       Int
  subtotal       Decimal @db.Decimal(12, 2)

  @@map("venta_items")
}
```

- `ventaId` es la **columna real** (la clave foránea `venta_id`).
- `venta` es un **campo de relación**: no existe en la base, le dice a Prisma cómo navegar de un ítem a su venta.
- `items VentaItem[]` es el lado inverso: permite pedir una venta "con sus ítems".
- `@relation(fields: [ventaId], references: [id])` significa "`venta_id` apunta al `id` de `ventas`". En MySQL se convierte en un `FOREIGN KEY`.

Cuando dos relaciones apuntan al mismo modelo (en `cajas`, `abierta_por` y `cerrada_por` apuntan a `usuarios`), cada una lleva un nombre para distinguirlas: `@relation("CajaAbiertaPor", …)`.

4.4 Enums

Los estados con valores fijos se declaran como `enum`, que en MySQL se convierte en una columna `ENUM`:

```
enum MetodoPago {
  efectivo
  transferencia
  mercadopago
  debito
  credito
}

model Pago {
  id         Int        @id @default(autoincrement())
  metodo     MetodoPago
  referencia String?    @db.VarChar(100)
  // …

  @@unique([metodo, referencia])   // evita registrar dos veces el mismo pago
  @@map("pagos")
}
```

Si el código intenta guardar `metodo: 'cheque'`, Prisma lo rechaza antes de llegar a la base.

5. Generar el cliente

Cada vez que cambia el esquema, el cliente se tiene que regenerar para que refleje los modelos nuevos:

```
npx prisma generate
```

Esto escribe el código en `src/generated/prisma/`. Ese código no se edita a mano ni se sube a Git: se regenera en cada instalación y en cada deploy. `prisma migrate dev` también lo regenera solo.

6. Migraciones: cómo cambia la base sin perder datos

6.1 Qué es una migración

Es un archivo SQL con los cambios necesarios para pasar la base de un estado al siguiente: crear una tabla, agregar una columna, crear un índice. Las migraciones se aplican **en orden** y quedan en Git, así cualquier persona (o el servidor de la demo) puede reconstruir la misma base desde cero.

6.2 El ciclo de trabajo

1. Editar `schema.prisma` (por ejemplo, agregar el campo `paginas` a `Libro`).
2. Ejecutar:

```
npx prisma migrate dev --name agrega-paginas-a-libros
```

3. Prisma compara el esquema con la base, escribe `prisma/migrations/20261004_agrega-paginas-a-libros/migration.sql`, lo aplica y regenera el cliente.
4. Revisar el SQL generado y subirlo a Git junto con el cambio del esquema.

La tabla `_prisma_migrations` (la crea Prisma en la base) guarda qué migraciones ya se aplicaron.

6.3 La base "sombra"

`migrate dev` usa una base temporal, la **base sombra** (*shadow database*). La vacía, aplica ahí todas las migraciones desde cero y compara el resultado con el esquema para detectar diferencias o migraciones editadas a mano. En el proyecto es `entrelineas_shadow` (sección 3.2). **Nunca guardes datos ahí**, porque Prisma la borra en cada `migrate dev`. Esta base no se usa en producción.

6.4 Los tres comandos que importan

Comando	Cuándo	Qué hace
`prisma migrate dev`	Desarrollo	Crea la migración nueva, la aplica y regenera el cliente
`prisma migrate deploy`	Deploy (Railway/Render)	Aplica las migraciones pendientes. Nunca crea nuevas ni borra datos
`prisma migrate reset`	Solo desarrollo	**Borra toda la base** y la rehace desde las migraciones (el seed se corre después, aparte)

Reglas:

- Nunca editar una migración que ya se aplicó en otro entorno. Si hay un error, se hace otra migración que lo corrige.
- Si un cambio puede perder datos (borrar una columna, achicar un `VARCHAR`), Prisma avisa y pide confirmación. Leer el aviso.

7. Leer datos

Todas las operaciones se escriben con la forma `prisma.<modelo>.<operación>({ … })` y devuelven una promesa (por eso llevan `await`).

7.1 Buscar uno

```
// Un libro por slug, para la página /libros/[slug]
const libro = await prisma.libro.findUnique({
  where: { slug: 'el-aleph-jorge-luis-borges' },
});
// Devuelve el objeto, o null si no existe
```

`findUnique` solo acepta campos `@id` o `@unique`. Para cualquier otro filtro se usa `findFirst`.

7.2 Buscar muchos, con filtros y paginación

Así se resuelve el catálogo de `/libros?genero=novela&q=borges&pagina=2`:

```
const POR_PAGINA = 20;

const where = {
  activo: true,
  genero: { slug: 'novela' },             // filtro a través de la relación
  OR: [
    { titulo: { contains: 'borges' } },
    { autor: { contains: 'borges' } },
    { isbn: 'borges' },
  ],
};

const [libros, total] = await Promise.all([
  prisma.libro.findMany({
    where,
    orderBy: { titulo: 'asc' },
    skip: (2 - 1) * POR_PAGINA,           // salteá la página 1
    take: POR_PAGINA,                     // traé 20
  }),
  prisma.libro.count({ where }),
]);
```

Con la intercalación (*collation*) `utf8mb4_unicode_ci`, `contains` no distingue mayúsculas ni tildes: buscar "borges" encuentra "Borges".

7.3 Elegir qué columnas salen: `select`

`rules.md` exige que los endpoints públicos nunca devuelvan datos internos. Con `select` se piden solo los campos necesarios:

```
const libro = await prisma.libro.findUnique({
  where: { slug },
  select: {
    titulo: true, autor: true, precio: true, stock: true, sinopsis: true,
    genero: { select: { nombre: true, slug: true } },
  },
});
```

Todo lo que no está en `select` no sale de la base. Es la forma más segura de armar respuestas públicas.

7.4 Traer relaciones: `include`

```
// Una venta con sus ítems y pagos, para el admin
const venta = await prisma.venta.findUnique({
  where: { id: 15 },
  include: { items: true, pagos: true },
});
```

`include` agrega relaciones completas; `select` elige campo por campo. No se pueden usar los dos en el mismo nivel.

8. Escribir datos

```
// Crear
const genero = await prisma.genero.create({
  data: { nombre: 'Novela', slug: 'novela' },
});

// Modificar
await prisma.libro.update({
  where: { id: 7 },
  data: { precio: '18500.00' },
});

// Modificar con operaciones atómicas
await prisma.libro.update({
  where: { id: 7 },
  data: { stock: { decrement: 1 } },     // stock = stock - 1, en la base
});

// Crear una venta con sus ítems en una sola operación (escritura anidada)
await prisma.venta.create({
  data: {
    codigo: 'ENT-4821',
    total: '37000.00',
    items: {
      create: [
        { libroId: 7, titulo: 'El Aleph', precioUnitario: '18500.00', cantidad: 1, subtotal: '18500.00' },
        { libroId: 9, titulo: 'Ficciones', precioUnitario: '18500.00', cantidad: 1, subtotal: '18500.00' }
      ],
    },
  },
});
```

Sobre borrar: existe `delete`, pero en `ventas`, `pagos`, `cajas`, `movimientos_caja` y `facturas` **no se usa nunca**. Se anula con un `update` que completa `anuladoPor`, `anuladoEn` y `motivoAnulacion` (`rules.md`, sección 4).

9. Dinero: el tipo Decimal

Los montos están en la base como `DECIMAL(12,2)`, que guarda el valor exacto. En JavaScript, en cambio, `0.1 + 0.2` da `0.30000000000000004`, porque los números de JS (*float*) no son exactos.

Prisma devuelve los `Decimal` como un objeto especial (de la librería Decimal.js), no como un número común. La regla del proyecto es operar en **centavos enteros**:

```
// De la base a centavos
const centavos = Math.round(Number(libro.precio) * 100);   // 18500.00 → 1850000

// Cálculos siempre con enteros
const subtotal = centavos * cantidad;

// De centavos a la base: string con dos decimales
const paraGuardar = (subtotal / 100).toFixed(2);           // "37000.00"
```

Estas conversiones viven en un solo módulo (`src/lib/dinero.js`) para no repetirlas. Al guardar se pasa un string (`'37000.00'`), que Prisma convierte a `DECIMAL` sin pérdida.

10. Transacciones: todo o nada

10.1 Qué es una transacción

Una **transacción** agrupa varias operaciones para que se apliquen todas juntas o ninguna. Si al confirmar un pedido se descuenta el stock del primer libro y falla el segundo, la base no puede quedar a medias: se deshace todo (*rollback*). Es la propiedad que garantiza que la caja y el stock no se desordenen.

10.2 Transacción interactiva

Prisma ejecuta una función con un cliente especial, `tx`. Todo lo que se hace con `tx` va en la misma transacción. Si la función lanza un error, Prisma deshace todo:

```
// Confirmar un pedido: descuenta el stock de cada libro o no descuenta nada
await prisma.$transaction(async (tx) => {
  for (const item of venta.items) {
    const { count } = await tx.libro.updateMany({
      where: { id: item.libroId, stock: { gte: item.cantidad } },   // solo si alcanza
      data: { stock: { decrement: item.cantidad } },
    });
    if (count === 0) {
      throw new AppError(409, 'SIN_STOCK', `No hay stock suficiente de ${item.titulo}.`);
    }
    await tx.movimientoStock.create({
      data: { libroId: item.libroId, tipo: 'venta', cantidad: -item.cantidad, ventaId: venta.id },
    });
  }
  await tx.venta.update({
    where: { id: venta.id },
    data: { estado: 'confirmada', stockDescontado: true, confirmadaEn: new Date() },
  });
});
```

Por qué `updateMany` con condición y no "leer, verificar y restar": si la web y el mostrador venden el último ejemplar al mismo tiempo, los dos podrían leer `stock = 1`, los dos pasarían la verificación y el stock quedaría en `-1`. Con `updateMany` la condición `stock >= cantidad` y la resta ocurren en **una sola instrucción SQL**: MySQL bloquea la fila, así que la segunda venta encuentra `stock = 0`, `count` vale `0` y se rechaza. Es la regla "el stock nunca queda negativo" de `rules.md`.

Importante: dentro de la función se usa siempre `tx`, nunca `prisma`. Lo que se haga con `prisma` queda fuera de la transacción.

11. Errores de Prisma que vamos a manejar

Cuando falla una operación, Prisma lanza un error con un código:

Código	Qué pasó	Uso en el proyecto
`P2002`	Se violó un `@unique`	Transferencia con número de operación repetido, webhook duplicado, ISBN repetido
`P2025`	No existe el registro que se quería modificar	Devolver 404
`P2003`	Clave foránea inválida	Un `libroId` que no existe

El webhook de Mercado Pago es **idempotente** gracias a `@@unique([metodo, referencia])`. Si llega el mismo aviso dos veces, el segundo `create` falla con `P2002` y se ignora:

```
try {
  await tx.pago.create({ data: { metodo: 'mercadopago', referencia: String(pagoMp.id), /* … */ } });
} catch (error) {
  if (error.code === 'P2002') return;   // ya estaba registrado: no hacer nada
  throw error;
}
```

El middleware central de errores traduce estos códigos a `AppError` con mensajes en español, y nunca devuelve el error original al cliente.

12. Dónde se usa Prisma dentro de la API

Según `rules.md`, **solo la capa `repositories/` importa `prisma`**. Los servicios piden datos a los repositorios y no saben que existe Prisma:

```
routes → controllers → services → repositories → prisma → MySQL
```

```
// src/repositories/libros-repository.js
import { prisma } from '../db/prisma.js';

export function obtenerLibroPublicoPorSlug(slug) {
  return prisma.libro.findFirst({
    where: { slug, activo: true },
    select: { titulo: true, autor: true, precio: true, stock: true, sinopsis: true },
  });
}
```

Para las transacciones, los repositorios reciben `tx` como parámetro opcional (`db = prisma`), así el servicio puede encadenar varias operaciones dentro de un mismo `$transaction`.

13. Tests contra la base de prueba

Los tests de la API (Vitest + Supertest) usan la base `entrelineas_test`. Antes de cada archivo de tests, `test/setup.js` reemplaza `DATABASE_URL` por `TEST_DATABASE_URL`. Como `src/db/prisma.js` lee esa variable recién cuando se carga, todo el código (repositorios incluidos) queda apuntando a la base de prueba sin cambiar una línea. En la Fase 1, cuando haya tablas, se suma un script que aplica las migraciones a esa base antes de correr los tests.

14. Datos de ejemplo (seed)

El **seed** es un script que llena la base con datos de prueba: los 2 usuarios, los 8 géneros y los 60 libros de la Fase 1. Se registra en `prisma7.config.ts` (`migrations.seed`) y se ejecuta con `npx prisma db seed`.

Desde Prisma 7, `migrate dev` y `migrate reset` ya no lo corren solos: después de rehacer la base hay que ejecutarlo a mano. Usa `upsert` (crear o actualizar), así se puede correr varias veces sin duplicar datos.

15. Prisma Studio

```
npx prisma studio
```

Abre en el navegador una tabla editable de cada modelo. Sirve para revisar qué guardó un endpoint o cargar un dato a mano en desarrollo. No se usa contra la base de la demo.

16. Resumen de comandos

Comando	Para qué
`npm run db:check`	Probar la conexión a MySQL
`npm run db:migrate -- --name <nombre>`	Crear y aplicar una migración (`prisma migrate dev`)
`npm run db:generate`	Regenerar el cliente (`prisma generate`)
`npm run db:studio`	Abrir Prisma Studio
`npx prisma migrate deploy`	Aplicar migraciones en la demo
`npx prisma migrate reset`	Rehacer la base de desarrollo desde cero (borra todo)
`npx prisma format`	Ordenar y validar `schema.prisma`

17. Glosario

Término	Definición
ORM	Herramienta que traduce entre tablas de la base y objetos del lenguaje
Esquema	Descripción de todas las tablas, columnas y relaciones
Modelo	Una tabla vista desde Prisma (`Libro` ↔ `libros`)
Migración	Archivo SQL versionado que cambia la estructura de la base
Cliente generado	Código que Prisma escribe a partir del esquema para consultar la base
Driver adapter	Conector de JavaScript que habla con MySQL en nombre de Prisma
Pool de conexiones	Grupo de conexiones abiertas que se reutilizan entre pedidos
Clave foránea	Columna que apunta al `id` de otra tabla (`venta_id`)
Transacción	Grupo de operaciones que se aplican todas o ninguna
Rollback	Deshacer todo lo hecho dentro de una transacción que falló

Término	Definición
Idempotente	Operación que da el mismo resultado si se ejecuta una o varias veces
Seed	Script que carga datos de ejemplo
Base sombra	Base temporal que usa `migrate dev` para comparar estados
Mínimo privilegio	Darle a cada usuario solo los permisos que necesita
Type stripping	Función de Node 24 que ejecuta TypeScript quitando los tipos al cargar el archivo

Diseño

1. Idea central

Entrelíneas es una librería de barrio: estantes de madera, lomos de tela, papel blanco y tinta. La web tiene que sentirse como **recorrer un estante**, no como una tienda genérica.

El elemento memorable es el estante. En el catálogo, las portadas se muestran con su proporción real, apoyadas sobre una línea de estante común a cada fila (alineadas abajo, no arriba). Libros de distintas alturas, todos sobre la misma base, como en la librería. Todo lo demás es sobrio y silencioso: el protagonista es el libro.

Principios:

- La portada manda. Nada de fondos con degradé ni tarjetas con sombra alrededor de los libros.
- Tipografía de lectura para los títulos de libros; tipografía de interfaz clara para todo lo operativo.
- En el admin, densidad y velocidad: el vendedor tiene un cliente esperando.

2. Color

Nombre	Hex	Uso
Papel	`#F8F8F5`	Fondo general
Blanco	`#FFFFFF`	Superficies (formularios, paneles del admin)
Tinta	`#1A2333`	Texto principal, encabezados
Grafito	`#5B6170`	Texto secundario, metadatos (autor, editorial)
Verde tela	`#1F4A3D`	Color primario: botones, enlaces, foco, elementos activos
Ocre lomo	`#B07D24`	Acento escaso: la línea del estante, "últimos ejemplares"
Línea	`#DEDCD5`	Bordes y divisores
Error	`#A3332D`	Errores y anulaciones

Reglas:

- El ocre no se usa para texto chico (no alcanza contraste AA sobre papel). Va en la línea del estante, en íconos o en texto grande.
- Estados del admin: éxito en verde tela, advertencia en ocre, error en rojo. Siempre acompañados de texto o ícono, nunca solo color.

Modo oscuro (solo admin en el MVP, opcional): fondo `#12171F`, superficie `#1A212C`, texto `#E8E6E1`, secundario `#9BA1AD`, primario `#6FAF95`, ocre `#D9A441`, línea `#2C3542`.

3. Tipografía

Familia	Rol
Literata (Google Fonts)	Logo, títulos de página, títulos de libros, sinopsis
Public Sans (Google Fonts)	Interfaz: navegación, botones, formularios, tablas, precios

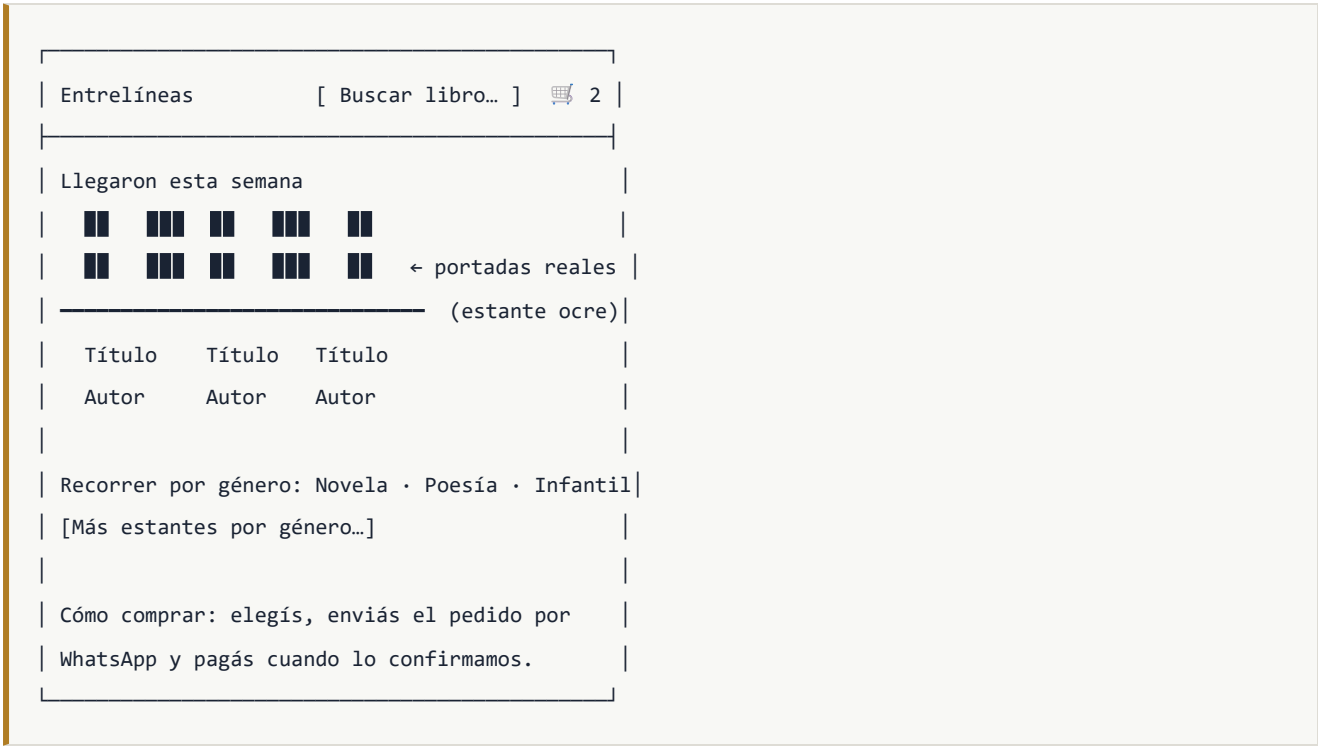
- Literata fue diseñada para lectura de libros, y en títulos de obras se nota. Public Sans es neutra y muy legible en formularios y tablas.
- Escala (ratio 1,25): 14 · 16 · 20 · 25 · 31 · 39 px. Texto base 16 px.
- Interlineado: 1,5 en Public Sans; 1,6 en Literata para sinopsis.
- Precios y montos con números tabulares (`font-variant-numeric: tabular-nums`) para que las columnas de la caja se alineen.
- Ancho de línea máximo de la sinopsis: 68 caracteres.
- Sin mayúsculas sostenidas en etiquetas. Todo en tipo oración.

Logo: la palabra "Entrelíneas" en Literata, con una línea ocre fina que pasa por debajo de las letras, como un renglón. Es el mismo trazo que el estante del catálogo.

4. Layouts

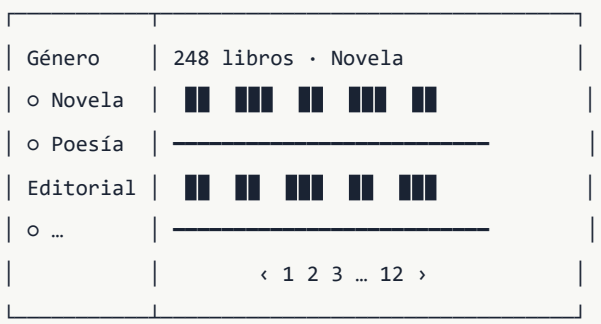
Inicio

Arranca directamente con un estante de novedades: el primer contacto es con libros, no con un banner.

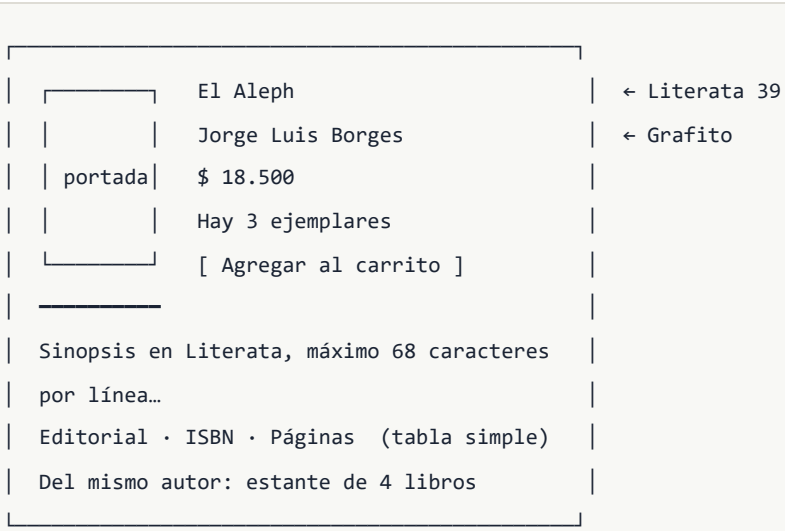


Listado (`/libros`)

Filtros a la izquierda en escritorio, en un panel desplegable en el celular. Estantes de 5 libros en escritorio, 2 en celular. Texto alineado a la izquierda.



Detalle de libro



Carrito

Una columna. Lista de libros, total, datos del cliente (nombre, teléfono, email opcional), casilla "Necesito factura" que despliega los datos fiscales, y un solo botón principal: **Enviar pedido por WhatsApp**. Debajo, una línea que explica qué pasa después: "Te vamos a responder por WhatsApp para confirmar stock y forma de pago."

Página del pedido (`/pedido/{token}`)

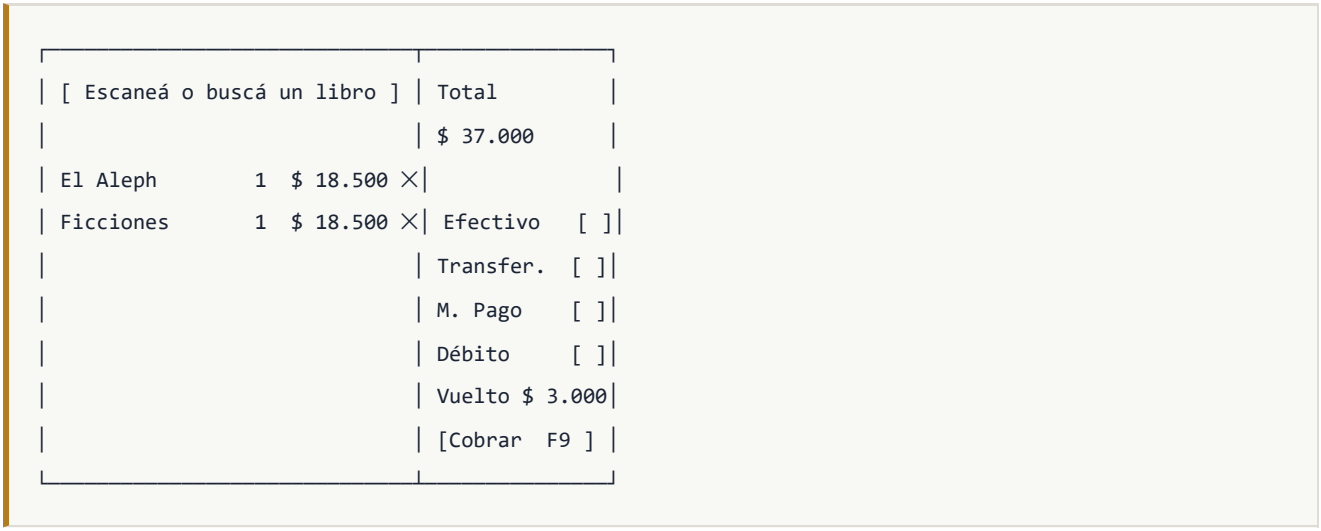
Encabezado con el código (`ENT-4821`) y el estado en palabras ("Confirmado, esperando el pago"). Debajo, el recorrido del pedido como una línea de pasos (acá sí corresponde una secuencia): Enviado → Confirmado → Pagado → Listo para retirar. Luego el detalle y las opciones de pago.

Admin — estructura

Menú lateral fijo: Inicio, Pedidos, Mostrador, Caja, Libros, Usuarios (solo admin). Arriba a la derecha: estado de la caja ("Caja abierta desde las 9:02") y usuario. Superficies blancas sobre papel, bordes de 1 px en color línea, radio de 4 px en campos y botones, 8 px en paneles.

Mostrador

Pensado para teclado y escáner. El campo de escaneo siempre tiene el foco.



Atajos: `F9` cobrar, `Esc` vaciar venta, `Supr` quitar el último libro.

Caja

Resumen arriba (efectivo, transferencia, Mercado Pago, débito, crédito; y web contra local) en una tabla con números tabulares alineados a la derecha. Debajo, la lista de movimientos del día. El cierre es un formulario corto: efectivo contado, observaciones, y la diferencia calculada en vivo.

5. Componentes

- **Botón primario:** fondo verde tela, texto blanco, radio 4 px. Uno solo por pantalla.
- **Botón secundario:** borde verde tela, fondo transparente.
- **Botón peligroso:** texto rojo con borde; para anular siempre pide motivo.
- **Campos:** borde línea, fondo blanco, foco con contorno verde tela de 2 px.
- **Portada:** sin sombra ni borde; si no hay imagen, un rectángulo verde tela con el título en Literata blanca.
- **Disponibilidad:** "Hay 3 ejemplares" en grafito; "Últimos ejemplares" con un marcador ocre; "Sin stock" en grafito y botón deshabilitado.
- **Estados de pedido:** texto con un punto de color a la izquierda, nunca solo un color.
- **Tablas del admin:** filas de 44 px, encabezado en Public Sans 14 seminegrita, montos alineados a la derecha.

6. Movimiento

- Un solo momento animado en el sitio público: al agregar un libro, el contador del carrito se actualiza con un cambio breve.
- En el admin, solo transiciones que confirman acciones (venta cobrada, caja cerrada).
- Todo se desactiva con `prefers-reduced-motion`.

7. Textos

- Botones con el verbo de lo que pasa: "Agregar al carrito", "Enviar pedido por WhatsApp", "Pagar con Mercado Pago", "Cerrar caja".
- La misma acción se llama igual en todo el flujo: el botón "Confirmar pedido" produce el aviso "Pedido confirmado".
- Errores que dicen qué pasó y cómo seguir: "No hay stock suficiente de El Aleph (quedan 1). Ajustá la cantidad."
- Estados vacíos que invitan a actuar: "Todavía no hay ventas hoy. Abrí la caja para empezar."

8. Tokens para Tailwind

Tailwind v4 (el que instala Next.js hoy) define los tokens en CSS con `@theme`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-papel: #F8F8F5;
  --color-tinta: #1A2333;
  --color-grafito: #5B6170;
  --color-tela: #1F4A3D;
  --color-ocre: #B07D24;
  --color-linea: #DEDCD5;
  --color-error: #A3332D;

  --font-serif: var(--font-literata), Georgia, serif;
  --font-sans: var(--font-public-sans), system-ui, sans-serif;

  --radius-campo: 4px;
  --radius-panel: 8px;
}
```

Con eso se usan clases como `bg-papel`, `text-tinta`, `bg-tela`, `font-serif`, `rounded-campo`. Las fuentes se cargan con `next/font/google` en `app/layout.js` y se exponen como las variables `--font-literata` y

`--font-public-sans`. Si el proyecto usara Tailwind v3, los mismos valores van en `theme.extend` de `tailwind.config.js`.

Reglas del proyecto

Reglas para cualquier persona o asistente de IA que escriba código en este repositorio. Ante una duda, priman este archivo, `arquitectura.md` y `PRD.md`, en ese orden.

1. Generales

- Leer `memory.md` y `task.md` antes de empezar una tarea, y actualizarlos al terminarla.
- Hacer solo lo que pide la tarea. Si aparece algo fuera del MVP, anotarlo en `task.md` (etapa 2) en vez de implementarlo.
- No agregar dependencias sin justificarlo en `memory.md`.
- Cambios chicos y verificables. Cada tarea termina con el código funcionando.

2. Idioma y nombres

- **Dominio en español** (coherente con el negocio): `venta`, `pago`, `caja`, `libro`, `obtenerVentaPorToken()`.
- Términos técnicos estándar en inglés cuando son de la librería o del framework (`req`, `res`, `router`, `middleware`).
- Variables y funciones en `camelCase`; componentes React en `PascalCase`; archivos de componentes `PascalCase.jsx`; resto en `kebab-case.js`.
- Tablas y columnas en `snake_case` y en plural para tablas.
- Textos de la interfaz en español rioplatense. Botones con verbo en infinitivo que dice exactamente qué pasa ("Agregar al carrito", "Enviar pedido por WhatsApp"); mensajes al usuario con voseo ("Revisá el teléfono").

3. Backend (Express)

- Capas: `routes → controllers → services → repositories`. Un controller no consulta la base; un repository no tiene lógica de negocio.
- Toda entrada se valida con Zod en `validators/` antes de llegar al controller.
- Errores con una clase `AppError(status, codigo, mensaje)` y un middleware central. Nunca devolver stack traces al cliente.
- Respuestas JSON con forma consistente: `{ data }` en éxito, `{ error: { codigo, mensaje } }` en error.
- Las integraciones externas viven solo en `integrations/`. Ningún service llama a `fetch` de un tercero directamente.

4. Dinero, stock y caja (reglas críticas)

- Montos como `DECIMAL(12,2)` en la base. En JavaScript, operar en **centavos enteros** (`Math.round(monto * 100)`) y convertir solo para mostrar. Nunca sumar floats directamente.
- **El precio lo decide el servidor.** Al crear un pedido o venta se ignoran precios que mande el cliente y se leen de la base.
- Toda operación que toque stock, pagos o caja va **dentro de una transacción**.
- El stock nunca queda negativo: el descuento se hace con una condición (`stock >= cantidad`) dentro de la transacción y se verifica cuántas filas se actualizaron.
- Cada cambio de stock genera un registro en `movimientos_stock`.
- **Nada se borra** en `ventas`, `pagos`, `cajas`, `movimientos_caja` ni `facturas`. Se anula con usuario, fecha y motivo.
- Una caja cerrada no se modifica.
- Un pago se asocia a la caja abierta **en el momento del cobro**.
- **Una importación nunca escribe sin vista previa confirmada**, y solo pisa los campos que el admin eligió.
- En importaciones, el ISBN siempre se lee y se guarda como **texto** (Excel suele convertirlo a número y romperlo).

5. Mercado Pago

- El webhook responde `200` de inmediato y procesa después.
- Validar la firma del webhook (`x-signature`) antes de procesar.
- Nunca usar el contenido del webhook como verdad: consultar siempre el pago por id a la API.
- Procesamiento idempotente: si ya existe un pago con esa referencia, no hacer nada.
- Las `back_urls` solo muestran mensajes; nunca cambian estados.
- Credenciales de prueba en la demo. El access token jamás llega al frontend.

6. Frontend (Next.js)

- Server Components por defecto. `"use client"` solo en componentes con estado, eventos o APIs del navegador.
- El catálogo se genera en el servidor con `revalidate`. `generateMetadata` en cada página de libro y de listado.
- Datos del admin: desde Client Components contra `/api/*` (la cookie viaja sola), o desde Server Components reenviando la cookie con `cookies()`.
- El carrito vive en un contexto de cliente y se guarda en `localStorage` envuelto en `try/catch`.
- Imágenes de portada siempre desde Cloudinary con transformaciones (`f_auto,q_auto,w_*`) y `alt` con el título del libro.
- Tailwind con los tokens definidos en `design.md`. No usar colores sueltos fuera de la paleta.

7. Seguridad

- Secretos solo en `.env` (nunca en el repo). Mantener `.env.example` actualizado sin valores reales.
- `bcrypt` con costo 12 para contraseñas.
- Rate limit en login y en creación de pedidos.
- Turnstile validado en el servidor.
- Tokens públicos de pedido generados con `crypto.randomBytes`, nunca con ids secuenciales.
- Endpoints públicos nunca devuelven datos internos (usuario, caja, costo, datos de otros pedidos).
- Archivos subidos: validar tipo y tamaño (imágenes y PDF, máximo 5 MB).

8. Accesibilidad

- Contraste AA mínimo, foco visible siempre, `label` en todos los campos.
- El mostrador se tiene que poder usar entero con teclado: el foco vuelve al campo del escáner después de cada acción.
- Respetar `prefers-reduced-motion`.

9. Tests

- Obligatorios para: cálculo de cierre de caja, procesamiento del webhook (incluido duplicado), descuento y devolución de stock, creación de pedido con precios manipulados.
- Vitest para unidades y Supertest para endpoints, contra una base de prueba.
- Los tests no llaman a servicios externos: se simulan las integraciones.

10. Git

- Ramas `feat/...`, `fix/...`, `docs/...`.
- Commits en español, en presente e imperativo: `agrega cierre de caja con arqueo`.
- Nunca commitear `.env`, claves ni certificados (`*.key`, `*.crt`).

Tareas

Marcar con `[x]` al terminar. Cada fase termina con algo que se puede probar. No pasar a la siguiente con la anterior rota.

Fase 0 — Preparación

- [x] **Decidir acceso a datos: Prisma o `mysql2` con SQL.** Registrar la decisión en `memory.md`. → Prisma 7.10.
- [x] Escribir `docs/guia-prisma.md` (conexión, esquema, migraciones, consultas, transacciones).
- [x] Subir el repositorio a GitHub (rama `main` + rama por fase).
- [x] Actualizar a Node.js 24 LTS (necesario para el cliente de Prisma 7).
- [x] Crear repositorio con `apps/web`, `apps/api` y `docs/` (copiar estos documentos).
- [x] `apps/api`: Express, estructura de carpetas, middleware de errores, Zod, `.env.example`.
- [x] `apps/web`: Next.js (App Router, JavaScript), Tailwind, fuentes Literata y Public Sans, tokens de `design.md`.
- [x] Rewrite de `/api/*` hacia Express en `next.config.js`.
- [x] MySQL local y conexión desde la API.
- [x] Configurar Vitest y Supertest con base de prueba.

Fase 1 — Base de datos y usuarios

- [ ] Crear todas las tablas de `arquitectura.md` (migraciones).
- [ ] Script de datos de ejemplo (seed): 2 usuarios, 8 géneros, 60 libros con portadas.
- [ ] Login, logout y `/api/auth/me` con JWT en cookie `httpOnly`.
- [ ] Middlewares `requireAuth` y `requireRole`.
- [ ] Pantalla `/admin/login` y layout del admin con menú lateral.
- [ ] ABM de usuarios (solo admin).

Fase 2 — Libros y catálogo

- [ ] Integración Cloudinary (subir desde URL, armar URLs con transformaciones).
- [ ] Integración Google Books con respaldo de Open Library (`buscarDatosPorIsbn`).
- [ ] Admin: alta de libro escaneando ISBN, con autocompletado y revisión.
- [ ] Admin: listado, edición de precio y stock (con movimiento de stock), activar y desactivar.
- [ ] Generación de slug único.
- [ ] Endpoints públicos de libros y géneros (sin datos internos).
- [ ] Página `/libros` con búsqueda, filtros en la URL y paginación.
- [ ] Componente estante (portadas alineadas abajo sobre la línea ocre).

- [ ] Página `/libros/[slug]` con `generateMetadata` y Open Graph.
- [ ] Página de inicio con estante de novedades y géneros.
- [ ] `sitemap.js` y `robots.js`.
- [ ] **Importación desde Excel:**
  - [ ] Conseguir (o simular) el Excel real de la librería y anotar su formato en `memory.md`.
  - [ ] Módulo `importacion/`: lector (ExcelJS), detección de columnas, normalización (ISBN, precio, stock) y validación.
  - [ ] Tests de normalización: ISBN con guiones, en formato científico, ISBN-10, precio con `$` y separadores, filas vacías, ISBN repetidos.
  - [ ] Tabla `importaciones` y tipo `importacion` en `movimientos_stock`.
  - [ ] Endpoints de plantilla, vista previa, confirmación, historial y descarga de errores.
  - [ ] Pantalla `/admin/libros/importar`: subir archivo, mapear columnas, vista previa, elegir modo y campos, confirmar, ver informe.
  - [ ] Procesamiento por lotes en transacciones.
  - [ ] Job opcional de enriquecimiento (datos y portadas desde APIs de libros).
  - [ ] Script `scripts/importar-excel.js` para línea de comandos (usarlo para cargar la demo).
- [ ] Probar la vista previa del link en WhatsApp.

Fase 3 — Carrito y pedidos

- [ ] Contexto de carrito con persistencia en `localStorage`.
- [ ] Página `/carrito` con datos del cliente y opción de factura.
- [ ] Cloudflare Turnstile en el frontend y validación en la API.
- [ ] `POST /api/public/pedidos`: precios desde la base, código `ENT-XXXX`, token, vencimiento.
- [ ] Armado del mensaje y apertura de `wa.me`.
- [ ] Rate limit en creación de pedidos.
- [ ] Job de vencimiento de pedidos pendientes.
- [ ] Admin: listado de pedidos con filtros y búsqueda.
- [ ] Confirmar (descuenta stock en transacción), cancelar (devuelve stock), entregar.
- [ ] Página pública `/pedido/[token]` con estado y línea de pasos.
- [ ] Test: pedido con precios manipulados usa los de la base.
- [ ] Test: confirmar sin stock suficiente falla sin descontar nada.

Fase 4 — Pagos

- [ ] Integración Mercado Pago: crear preferencia y consultar pago.
- [ ] Botón "Pagar con Mercado Pago" en la página del pedido.
- [ ] Webhook: respuesta inmediata, validación de firma, registro en `webhook_eventos`, consulta del pago, registro idempotente.
- [ ] Probar con túnel (ngrok o Cloudflare Tunnel) y tarjetas de prueba.

- [ ] Transferencia: datos de pago en la página del pedido y subida de comprobante.
- [ ] Admin: registrar pago por transferencia con número de operación único.
- [ ] Anulación de pagos (solo admin, con motivo).
- [ ] Cambio automático de estado a `pagada` cuando los pagos cubren el total.
- [ ] Test: webhook duplicado registra un solo pago.
- [ ] Test: transferencia con número de operación repetido es rechazada.

Fase 5 — Mostrador y caja

- [ ] Abrir caja con efectivo inicial (una sola abierta a la vez).
- [ ] Indicador de caja en el encabezado del admin.
- [ ] Mostrador: campo de escaneo con foco permanente, búsqueda por texto, alta rápida si el ISBN no existe.
- [ ] Cobro con uno o varios métodos y cálculo de vuelto.
- [ ] `POST /api/admin/ventas` en una transacción (venta, items, pagos, stock, movimientos).
- [ ] Atajos de teclado del mostrador.
- [ ] Ingresos y egresos manuales.
- [ ] Asociar a la caja que se abre los pagos web que llegaron sin caja.
- [ ] Cierre con arqueo y diferencia calculada en vivo.
- [ ] Resumen por método y por canal; historial de cajas.
- [ ] Test: cálculo de efectivo esperado con ventas, ingresos, egresos y pagos anulados.
- [ ] Test: no se puede modificar una caja cerrada.

Fase 6 — Dashboard y facturas

- [ ] `GET /api/admin/dashboard` con ventas del día y del mes, métodos, canales, más vendidos y pendientes.
- [ ] Pantalla de inicio del admin.
- [ ] Subida de PDF de factura a una venta.
- [ ] Botón "Descargar factura" en la página pública del pedido.

Fase 7 — Calidad y publicación

- [ ] Revisión de accesibilidad (teclado, contraste, foco, `alt`).
- [ ] Revisión de seguridad con la lista de `rules.md`.
- [ ] Deploy de la web en Vercel y de la API con MySQL en Railway o Render.
- [ ] Webhook de Mercado Pago apuntando a la URL de producción de la demo.
- [ ] Usuario de demo y datos de ejemplo cargados.
- [ ] README: problema, capturas o GIF, stack, diagrama del flujo, cómo correrlo, credenciales de demo, tarjeta de prueba, decisiones técnicas y próximas funcionalidades.

Etapa 2 (después del MVP)

- [ ] ARCA en homologación: certificado (WSASS), WSAA con token cacheado, WSFEv1, CAE, PDF con QR.
- [ ] QR de Mercado Pago integrado en el mostrador.
- [ ] Point integrado (si hace falta).
- [ ] Encargos de libros sin stock.
- [ ] Historial de clientes por teléfono.
- [ ] Alertas de stock bajo, proveedores y compras.
- [ ] Promociones, descuentos y devoluciones.
- [ ] Exportar reportes a Excel y PDF.
- [ ] Notificaciones por email.

Memoria del proyecto

Contexto acumulado para retomar el trabajo (persona o asistente de IA). Leer al empezar cada sesión. Actualizar al terminar: decisiones nuevas, problemas encontrados y estado.

Contexto

- Proyecto de **portfolio**. La librería "Entrelíneas" es ficticia.
- Objetivo de la demo: que quien la revise pueda pedir, pagar con Mercado Pago de prueba y usar el mostrador y la caja en pocos minutos.
- Documentos de referencia: `PRD.md` (qué), `arquitectura.md` (cómo), `rules.md` (reglas), `design.md` (visual), `task.md` (avance).

Decisiones tomadas

Tema	Decisión	Motivo
Login de clientes	No hay. El pedido se valida por WhatsApp con un código	Menos fricción; el número de WhatsApp identifica al cliente
Checkout	Pedido guardado como `pendiente` + enlace `wa.me`	No requiere la API de WhatsApp Business
Antibots	Cloudflare Turnstile + rate limit + vencimiento a 2 h	Proteger sin pedir cuenta
Pago online	Mercado Pago Checkout Pro con webhook	Confirmación automática; no se manejan tarjetas
Transferencias	Número de operación único + comprobante; confirmación manual	No se pueden verificar automáticamente
Ventas del local	Misma tabla `ventas` con `canal = 'local'`	Una sola fuente para la caja
Pagos en la caja	Cuentan en la caja abierta al momento del cobro	La caja refleja lo cobrado ese día
Borrados	Ningún pago, venta, caja ni factura se borra; se anula	Historial confiable
Escáner	Lector USB como teclado, ISBN-13 como identificador	Sin drivers ni librerías
Datos de libros	Google Books, con Open Library de respaldo	Mejor cobertura en español
Imágenes	Cloudinary; se guarda `public_id`, no la URL	Transformaciones y cambio de proveedor fácil

Tema	Decisión	Motivo
Frontend	Next.js (App Router)	SEO y vista previa de links en WhatsApp
Backend	Express + MySQL	Stack ya conocido
Web y API	Mismo dominio vía rewrites de Next	Cookies de sesión sin CORS entre dominios
Lenguaje	JavaScript	Coherencia con proyectos anteriores
Node.js	24 LTS (`engines` en la raíz)	Node 20 quedó sin soporte (abril 2026) y Node 24 ejecuta el cliente TypeScript de Prisma 7 sin compilar
Bases MySQL	`entrelineas` , `entrelineas_shadow` y `entrelineas_test` , con el usuario `entrelineas` que solo tiene permisos sobre esas tres	Mínimo privilegio: en el mismo servidor hay otras bases de otros proyectos
Monorepo	npm workspaces (`apps/api` , `apps/web` , `docs/pdf`)	Un solo `npm install` y un solo lockfile
Facturas	MVP: carga manual de PDF. Etapa 2: ARCA en homologación	Terminar el MVP primero
QR y Point de Mercado Pago	MVP: registro manual. Etapa 2: integración	Reutilizar la lógica del webhook después
Diseño	Paleta papel, tinta, verde tela y ocre; Literata + Public Sans; catálogo como estante	Identidad propia de librería
Importación desde Excel	En el MVP, con vista previa obligatoria y modo explícito (crear, actualizar o ambos)	La librería ya tiene su catálogo en Excel; nada se pisa sin que el admin lo vea
Acceso a datos	Prisma ORM 7.10 (estable) con `@prisma/adapter-mariadb`	Migraciones versionadas, transacciones simples y consultas seguras. La etiqueta `latest` de npm apunta a Prisma 8 en versión candidata (rc), por eso se fija la 7.10. Guía completa en `guia-prisma.md`
Repositorio y ramas	GitHub desde el inicio. `main` siempre estable; una rama por fase (`feat/fase-0-preparacion` , …) que entra a `main` por pull request	Historial prolijo para el portfolio; Vercel publica `main` y genera vistas previas de cada rama
Librería de Excel	ExcelJS (`.xlsx` y `.csv`); los `.xls` viejos se guardan como `.xlsx` antes de subir	Mantenida y disponible en npm; el paquete `xlsx` de npm quedó desactualizado

Decisiones pendientes

- **Formato del Excel de la librería:** qué columnas tiene, cómo vienen los precios y si incluye stock. Ajustar el mapeo automático cuando se conozca.

Notas técnicas para recordar

- Mercado Pago: nunca confiar en el cuerpo del webhook ni en las `back_urls`; consultar el pago por id.
- MySQL 9 + driver `mariadb`: las URLs llevan `?allowPublicKeyRetrieval=true`; sin eso Prisma muestra `pool timeout`.
- El puerto 4000 lo usa también otro proyecto local (bonhomia). Si está corriendo, levantar la API con `PORT=4100` y la web con `API_URL=http://localhost:4100`.
- npm 11 bloquea los scripts de instalación: los de `prisma`, `@prisma/engines` y `unrs-resolver` están aprobados en `allowScripts` del `package.json` raíz. Si se actualiza Prisma, aprobar la nueva versión con `npm approve-scripts`.
- Disco C: muy justo de espacio: después de instalar dependencias, correr `npm cache clean --force`.
- Para probar el webhook en local hace falta un túnel; la URL cambia en cada ejecución del plan gratuito, así que va en `.env`.
- ARCA (etapa 2): el WSASS se adhiere con la clave fiscal de persona física desde el Administrador de Relaciones; el token del WSAA se cachea hasta su vencimiento.
- Revisar la documentación vigente de Mercado Pago y de ARCA al empezar cada integración: sus APIs cambian.

Estado

- Fase actual: **0 — Preparación, completa.** Próxima: Fase 1 (rama `feat/fase-1-base-de-datos`).
- Última actualización: 2026-10-04 — Fase 0 terminada: API y web funcionando, conexión a MySQL, tests pasando y repositorio en GitHub.

Registro de sesiones

- 2026-10-04 — Lectura de documentos, PDF de documentación, `bitacora.md` — decidir acceso a datos.
- 2026-10-04 — Prisma elegido, `guia-prisma.md`, Git con rama `feat/fase-0-preparacion`, Next.js creado en `apps/web` — liberar disco, instalar dependencias, contraseña de MySQL, repositorio en GitHub.
- 2026-10-04 — Espacio liberado, Node 24, bases y usuario MySQL, API Express + Prisma, web Next.js con tokens, rewrite, Vitest (6 tests), push a GitHub — empezar la Fase 1.